VAST Builders Challenge

VAST Builders Challenge: Video Agents

> 💡 Before today, see BEFORE_YOU_BUILD.md for what to do in advance. If you want the PDF version, click here.

If you want the PDF version of the Build Day guide, click here.

Spend the day building with video:



The infrastructure is already running, so you skip straight to the interesting part: turning hours of video into something that searches, reasons, and acts.

1. Kickoff

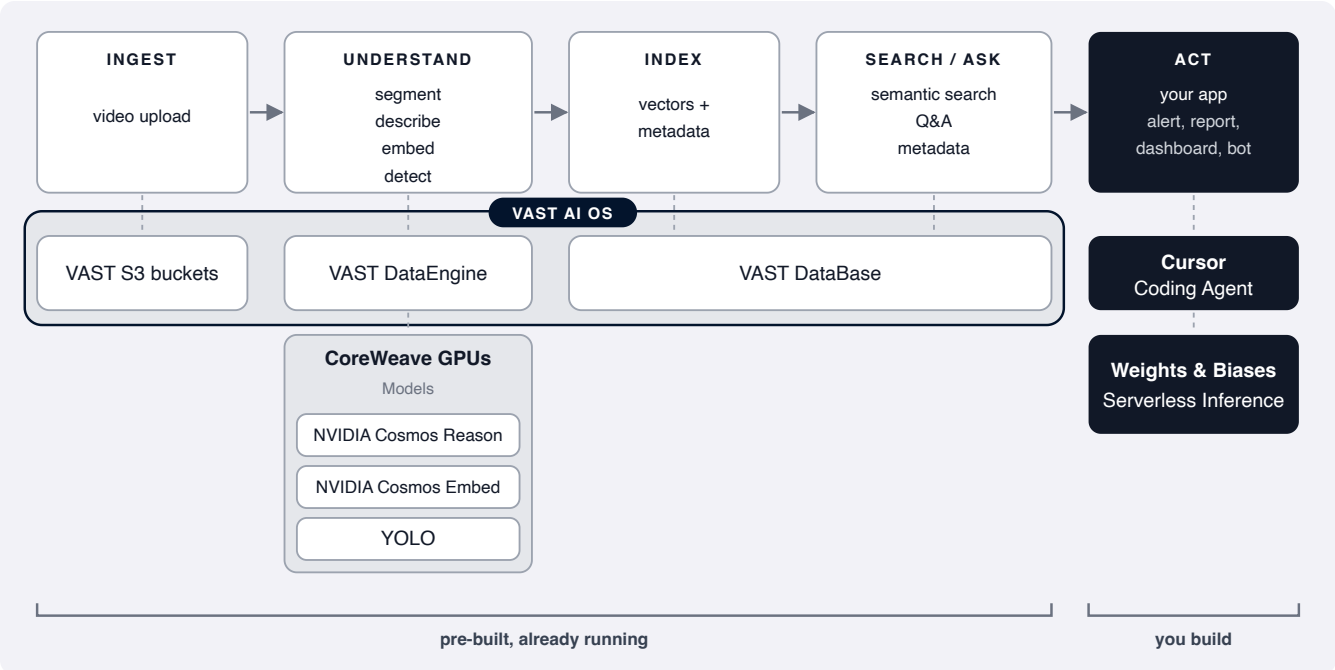
What you're building

An app that understands video and does something with it. Search hours of footage in plain language, ask what happened, detect and track objects, or call an action when something matters.

Pick one idea and ship a working app by end of day.

The Builders Stack

Video runs through a pipeline that understands and indexes it:



Everything under "pre-built, already running" is done for you. The pipeline ingests, understands, and indexes video, and the models it calls are already deployed and serving. You build something cool that searches and acts.

> 💡 For a deeper dive, check out the Architecture Reference.

2. Access Infra

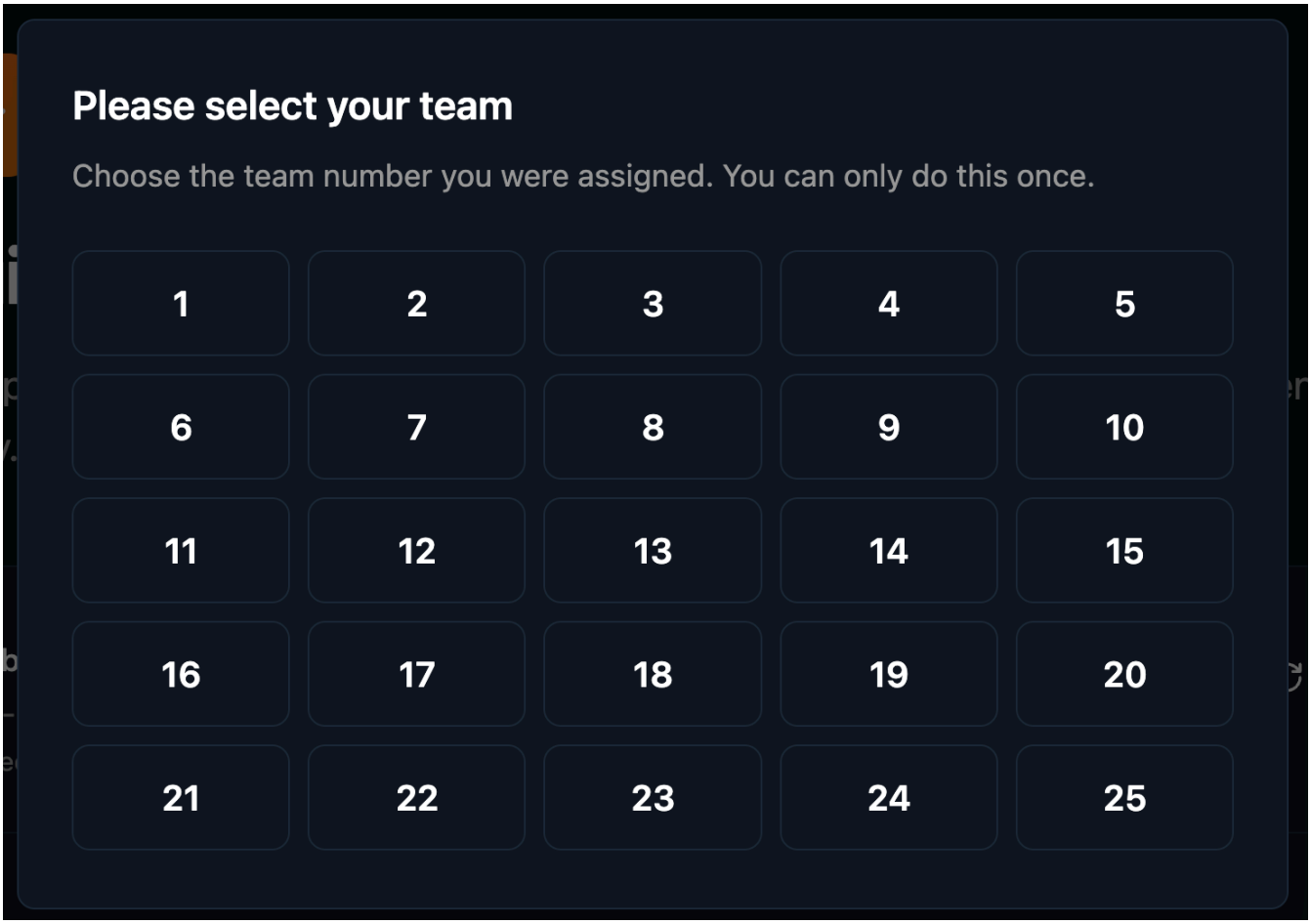
👉 Visit and join Cosmos to ask questions and access the Infra! (view PDF guide for infra onboarding)

You run Cursor on your laptop and connect to the workshop VM over SSH. Commands and files live on the VM, not your laptop.

Your team

Form your team and sit together first, then select your assigned team number. Teammates can pair, build in parallel, or each connect to the same lab environment.

> Ensure you select the assigned team (e.g. `team-1`) so all team members access the same video ingestion pipeline.



You build as a team. Your team shares one video ingestion instance, one index, and one set of credentials, so anything a teammate ingests shows up in every team members searches.

Coding Agent

> 💡 All registrants who checked in received Cursor credits. Confirm yours at cursor.com/dashboard/usage, each request should show as `Free`. Didn't sign up for Cursor? Sign up, then request credits. Need more credits or a top-up? Use the

> same form.

Describe what you want in plain language and let the code agent build. That's how the skills are meant to be used.

You can run Cursor locally, but the infrastructure (the repo, skills, and your credentials) lives on the VM, so you need **Cursor Remote-SSH access** to your lab VM.

Follow the instructions on your lab page to log in, configure your laptop, and connect once from a terminal. Then load it in Cursor:

1. Open a new Cursor window and choose **Connect via SSH**.
2. Pick your host from the list (the `bc-<username>` name from the lab page).
3. Open the `vast-builders-challenge` folder.

> 💡 Set the model Cursor uses to **Auto** in the agent panel to save tokens.

Before you kick off the coding agent and start using skills, head to the next section. We'll circle back to Skills very soon.

> 💡 Want a guided run instead of reading ahead? Once Cursor is up, run `/build-day-quickstart` and it'll walk you through the Video Search & Summary UI, Skills, Test Drive, and Build one step at a time.

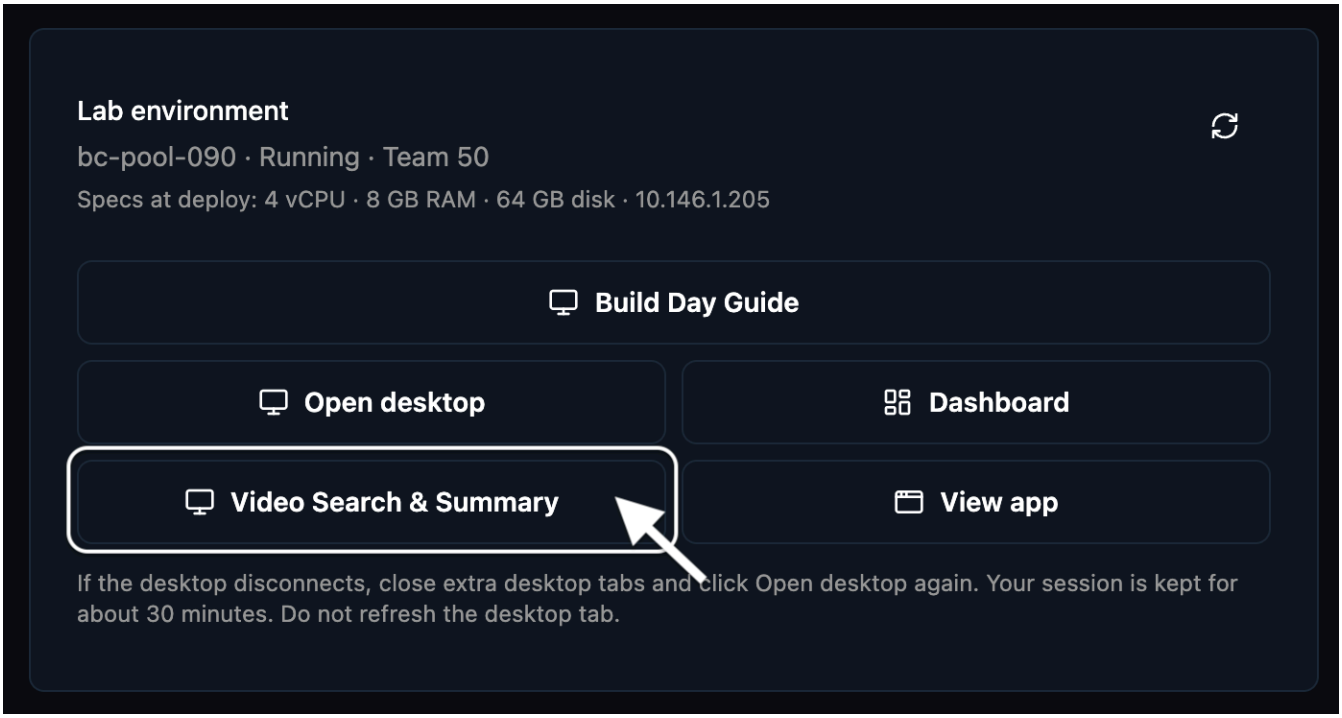
If something looks off

Run the health check in the Reference section. If it fails, or you need more help, run `/ask-cosmos`. It writes the note but does not post it, so **copy it and post it on Cosmos yourself**, and we'll follow up.

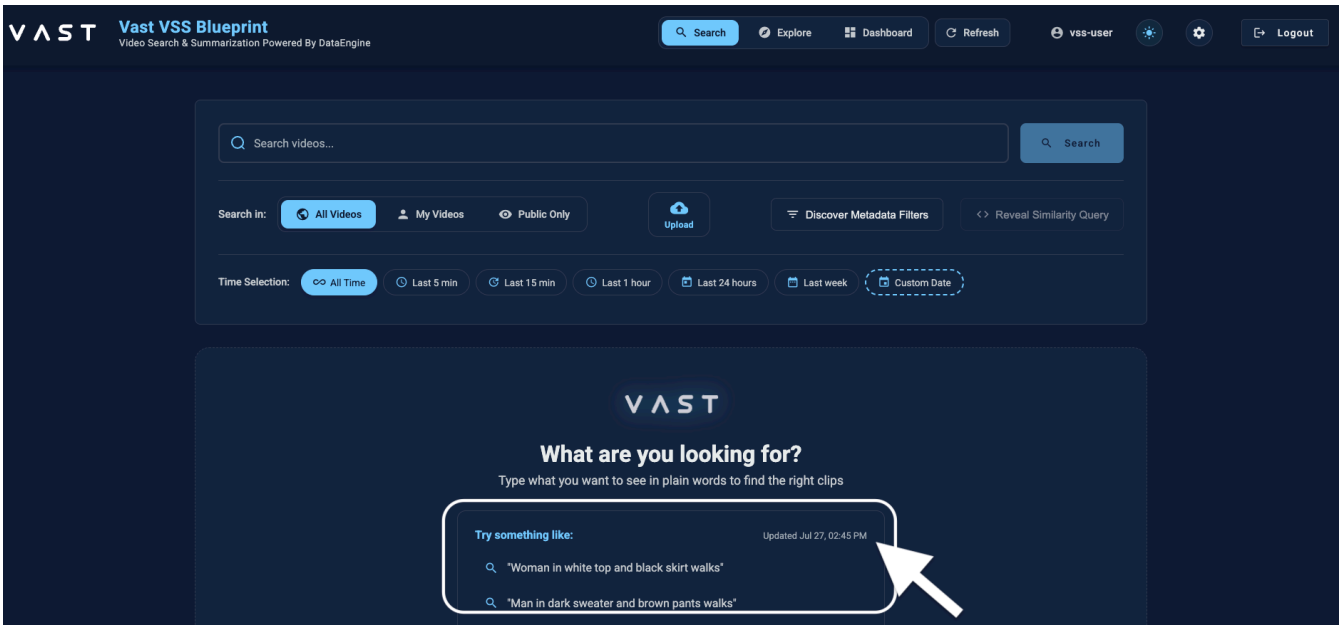
Video Search & Summary UI

> 💡 This example app runs on the same API as the skills (`.cursor/skills`) you will be using. It's a built idea of what you can build today, and a quick way to check things out while building.

From the same lab page, click on Video Search & Summary:



Try a few of the search suggestions to see ranked clips with timestamps and the description Cosmos Reason returned for each one:



Explore the three tabs: **Search** to query videos, **Explore** to browse what's indexed, and **Dashboard** to get stats.

3. Skills, Skills, Skills

> 💡 We'll try these out in the next section.

The core skills for the day are `ingest/` and `retrieval/`, in `.cursor/skills/`. More skills cover the rest of the deployed stack, if you want to explore. Each skill guides the coding agent: the endpoint, the request, the response, and what to do when it fails.

Describe what you want and the coding agent loads the matching skill.

```
"re-ingest the warehouse video with a prompt about safety gear"
"find people near the entrance after 6pm"
"summarize what happens in the warehouse video"
```

Skills read what they need from environment variables, so nothing should ask you for a password or a URL. If something isn't working, ask for help.

Re-ingest: running it again

> ⚠️ **IMPORTANT:** Avoid having everyone on the team re-ingest large chunks of video (i.e. hours of video). Designate 1-2 team members to handle kicking off majority of the re-ingestion. It's okay for each team member to re-ingest a few videos to try things out.

Your team's video is already indexed. Re-ingest means running it through the pipeline again with a different prompt, so the descriptions match what you're building.

Skill	Use it to
`reingest-videos`	Re-run a whole indexed video with a new prompt or metadata

Skill	Use it to
`reingest-chunk`	Re-run one specific chunk, found by filename, scene, date, or camera

Re-ingesting takes a few minutes. Every segment is described, embedded, and detected again before it becomes searchable. Ask Cursor to confirm before you go looking.

> 💡 **The prompt decides what gets indexed.** Cosmos Reason describes every segment following an ingestion prompt. Anything it doesn't ask about never gets written down, so you can't search for it later.
>
> On a construction site you might ask it to describe safety gear. In a public space you might ask whether anyone with an umbrella is in frame. Search the existing index for what your idea needs; if it isn't there, that's what re-ingesting is for.
>
> Tell Cursor what you want described, in your own words, or one of the built-in preset scenarios. Ask Cursor to list the scenarios (`list-metadata` looks these up live) if you want to see what's available. We'll try this out in the next section.
>
> Not clear? Flag down someone to clarify.

Retrieval: Searching Video

Skill	Use it to
`search`	Find moments matching a description, with filters for time, location, camera, tags.
`agent-qa`	Ask a question and get an answer with evidence, instead of a list of hits.
`videos`	Browse what's indexed, play a clip, read its captions and detections, summarize a whole video.
`dashboard`	Check what's indexed and whether ingest is healthy.
`suggest-prompts`	Get generated example queries and notable recent events.

Worth knowing: `vastdb-read` queries the database directly, for when you want to check the database contents.

4. Test Drive

Before you build, run one loop by hand. It tells you the whole stack is working and helps you understand what you can build.

Search what's there

Your team's index already has video in it, so start by looking:

```
what's in the index? show me a few examples
```

Then search for something specific to your idea:

```
find the moment where <something you care about> happens
```

You get back ranked segments with timestamps, scores, and the description the model wrote. Play one to confirm it's the moment you meant.

Ask instead of search

```
what happens in <one of those videos>?
```

Same index, different kind of answer. The first hands you moments. The second reads those moments and writes you an answer.

Do you want to show someone the clip, or tell them what happened? Most of designing a video app is picking one.

Find what's missing

Search for something your idea needs that the existing descriptions probably don't mention. Counts of people. What someone is carrying. Whether a vehicle stopped.

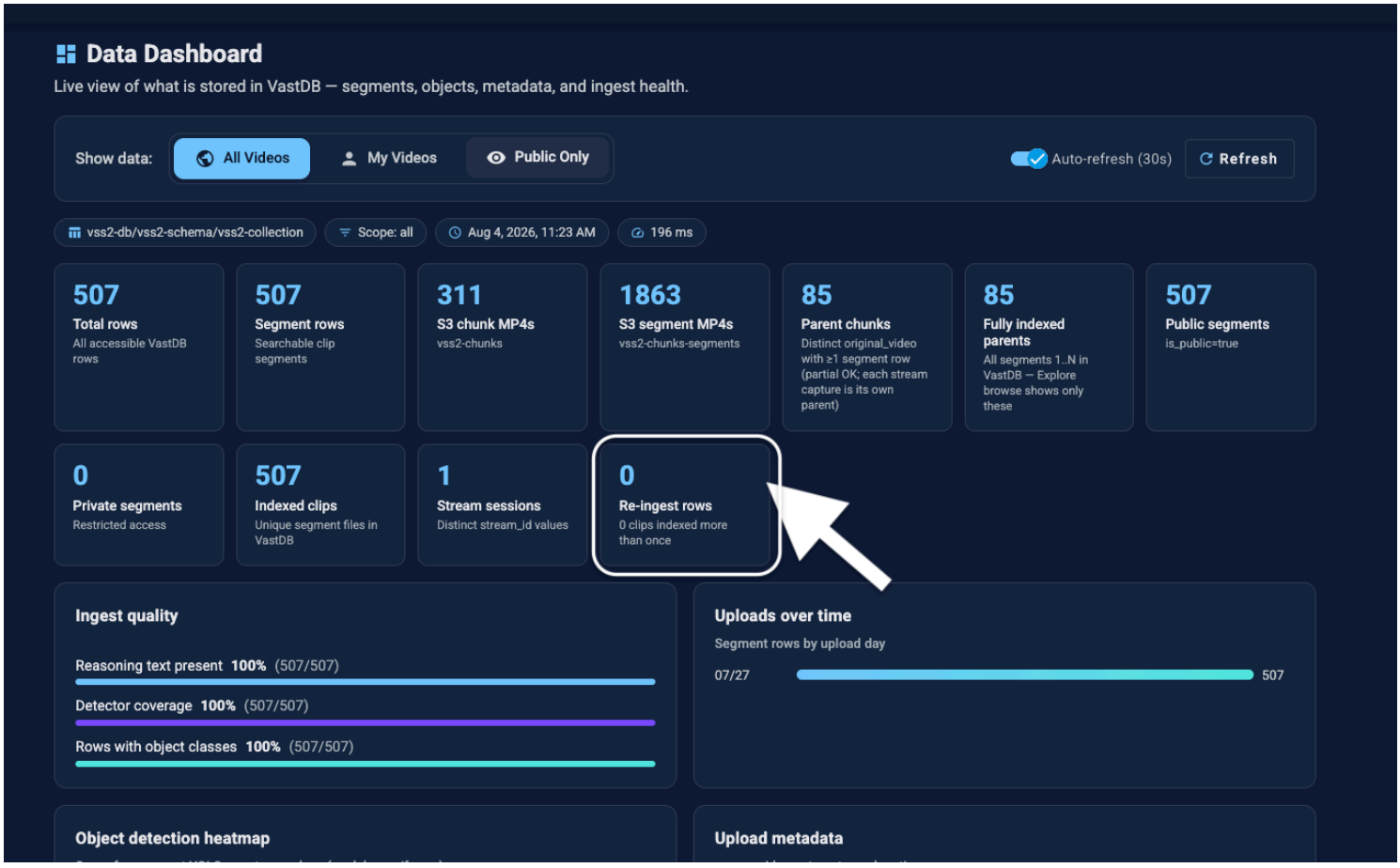
If it comes back empty, that's not a broken search. It means the ingestion prompt never asked about it, so nothing was written down.

Re-ingest with your prompt

```
re-ingest <that video> with a prompt that describes <what your app needs>
```

Cursor loads `reingest-videos`, shows you what it's about to re-run, and asks for the prompt. Give it a few minutes, then run the same search again. This time it matches.

To watch progress, ask `is it done yet?` or open the Dashboard tab:



> 💡 That gap, between what you searched for and what the prompt asked about, is the thing to keep in mind. Everything you can build depends on what the descriptions say.

That's the whole loop

Search, ask, re-ingest, search again. Everything you build today sits on those steps. If they worked, your stack is healthy and you can start building.

If any of them didn't, run the health check in the Reference section.

Recap: What you have

- A **video ingestion instance** running for your team: the pipeline ingests, understands, and indexes your video.
  - Cosmos Reason, Cosmos Embed, and YOLO run on CoreWeave GPUs; you don't call them directly, you query the vectors generated.
- A **lab VM you reach from Cursor over SSH**, with this repo and your credentials and endpoints available as environment variables.
- **Serverless LLM inference** from Weights & Biases by Coreweave for your app's own logic.
- A set of **skills** that drive the pipeline in plain language. See the section on Skills.

> ⚠️ **Don't ingest videos from the internet (e.g. YouTube).** We picked the provided video sources specifically because their licensing allows this use.

5. Build

What "done" looks like

You have a working index and you know how to query it. The rest of the day is what you build on top. A small app, agent, or a dashboard. A clear use case.

What video you have

The videos come from a few kinds of real-world footage:

- Dashcam driving footage of vehicles and pedestrians interacting at intersections and crossings
- Overhead, multi-camera highway footage tracking vehicle movement along a stretch of interstate
- A private neighborhood camera capturing car movement

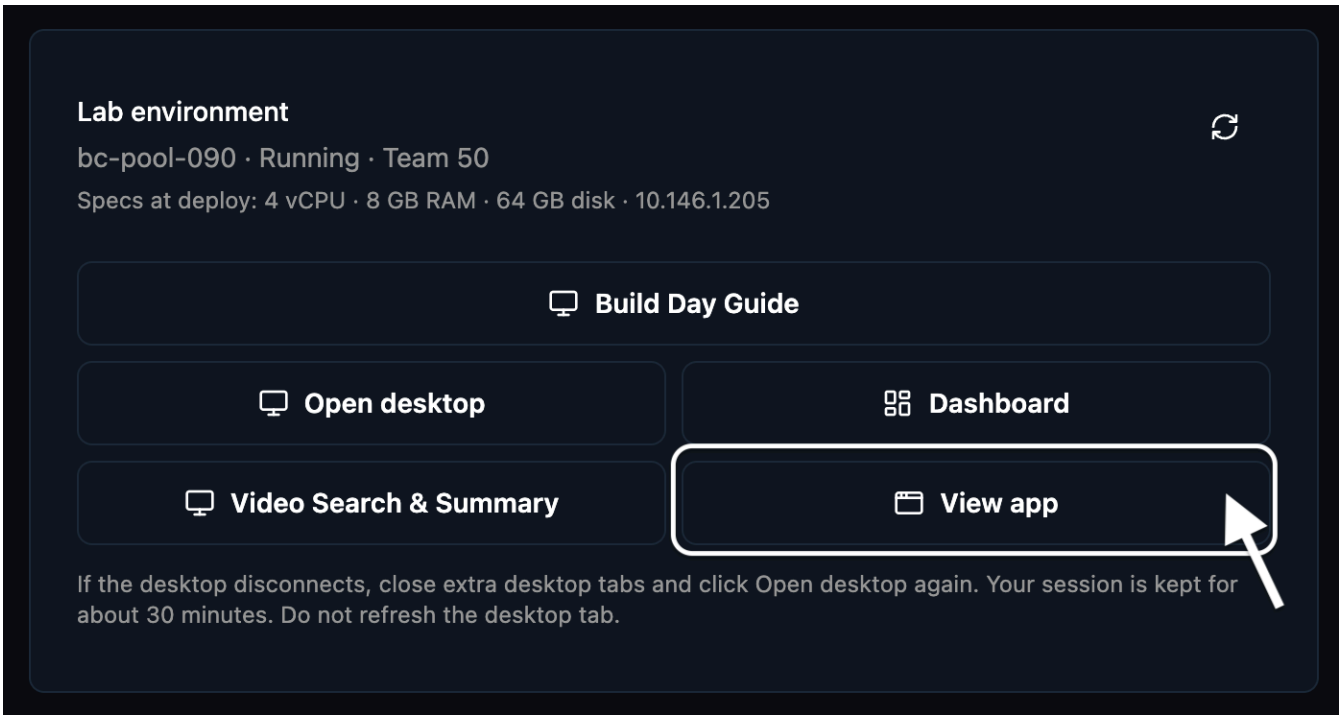
Explore what's actually indexed in the Video Search & Summary UI (previous section).

Your index is already full and searchable. For the full list of folders, camera IDs, use cases, and worked example queries, see the Architecture Reference's Video corpus section.

The loop

1. **Pick a use case**
2. **Build an app or agent**
3. **Deploy and iterate**

> 💡 Once you've deployed with `/deploy-app-no-registry`, check out your app in your own browser at workshop.thecosmoslabs.com — click **App**.



If the existing captions cover what you need, you never have to think about prompts. If they don't, re-ingest the footage with a different prompt.

> 💡 **Start with a few clips.** Read the captions that come back before you re-ingest anything at volume.

LLM access

Search and Q&A come from your VSS instance. Anything your app decides on top of that, classifying results, drafting a summary, choosing an action, runs on serverless LLM inference from Weights & Biases.

Your `WANDB_` keys are already in your environment. Point your own app's reasoning or agent at the inference endpoint using those. If you need more credits, reach out to the Weights & Biases by CoreWeave team :)

The skills used by Cursor can be used by agent frameworks too. They follow the standard `SKILL.md` format, so most frameworks load them straight from `.cursor/skills/`.

6. Demos

Submissions will start around 4:30pm. Instructions here: https://tokensand.com/vastlondon (click `Submit your project`).

We'll do a first round of judging with each team to walk through what you built and to hear how the day went.

Reference

Health check

If something isn't working, pull first. The repo is pre-cloned and may be behind. Type the following into Cursor:

```
run a git pull
```

Then ask Cursor:

```
check that everything is working
```

If it fails, or you need more help, run `/ask-cosmos`. The skill writes a note with the relevant details but **does not post it**. Copy it and post it on Cosmos yourself, and we'll follow up.

Your team's values

Everything the skills need is already in your environment. `config.example` in this repo lists every variable with a description.